



udp UNIVERSIDAD
DIEGO PORTALES

UNIVERSIDAD DIEGO PORTALES
ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

TALLER DE REDES

Tarea 2

Autor:
Carlos Beaudout

Profesor:

Junio 2026

Índice

1. Capítulo I: Introducción al protocolo RTSP	2
1.1. Historia del protocolo	2
1.2. Objetivo del protocolo	2
1.3. Funcionamiento del protocolo	2
1.4. Software servidor: MediaMTX	3
1.5. Software cliente: FFmpeg	3
1.6. Levantamiento del entorno	4
1.7. Descripción de los parámetros utilizados	5
1.8. Suposición de fallo	5
2. Capítulo II: Análisis de tráfico	6
2.1. Herramienta utilizada	6
2.2. Filtrado de tráfico	6
2.3. Análisis del tráfico RTSP	6
2.4. Comportamiento de la sesión	7
2.5. Posibles fallos en la comunicación	7
3. Conclusiones	8
4. Bibliografía	8

1. Capítulo I: Introducción al protocolo RTSP

1.1. Historia del protocolo

RTSP (Real Time Streaming Protocol) es un protocolo de la capa de aplicación definido por la IETF en 1998 (RFC 2326). Su objetivo es controlar la transmisión de contenido multimedia en tiempo real.

RTSP no transporta directamente el contenido multimedia, sino que se encarga de controlar las sesiones establecidas entre un cliente y un servidor.

1.2. Objetivo del protocolo

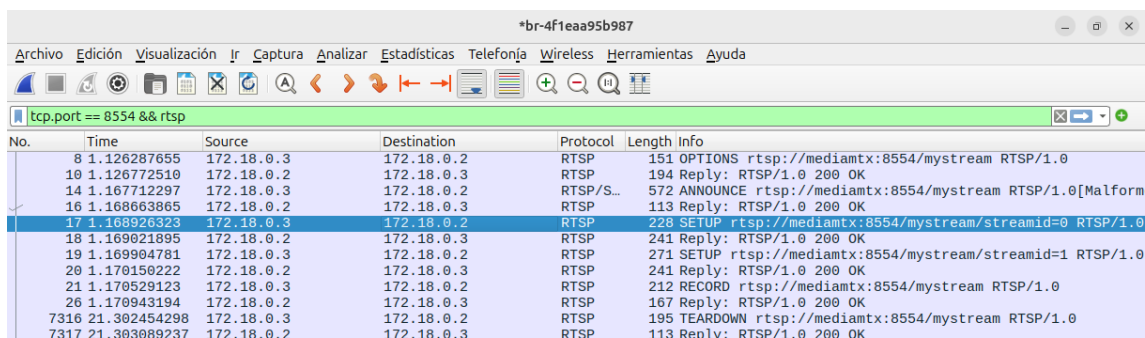
El objetivo de RTSP es permitir el control de sesiones multimedia, incluyendo el inicio, configuración y finalización de flujos de audio y video transmitidos en tiempo real.

1.3. Funcionamiento del protocolo

RTSP funciona bajo un modelo cliente-servidor mediante el intercambio de mensajes de control.

Durante el desarrollo del laboratorio fue posible observar los siguientes mensajes:

- OPTIONS
- ANNOUNCE
- SETUP
- RECORD
- TEARDOWN
- 200 OK



No.	Time	Source	Destination	Protocol	Length	Info
8	1.126287655	172.18.0.3	172.18.0.2	RTSP	151	OPTIONS rtsp://mediamtx:8554/mystream RTSP/1.0
10	1.126772510	172.18.0.2	172.18.0.3	RTSP	194	Reply: RTSP/1.0 200 OK
14	1.167712297	172.18.0.3	172.18.0.2	RTSP/S...	572	ANNOUNCE rtsp://mediamtx:8554/mystream RTSP/1.0[Malform
16	1.168663865	172.18.0.2	172.18.0.3	RTSP	113	Reply: RTSP/1.0 200 OK
17	1.168926323	172.18.0.3	172.18.0.2	RTSP	228	SETUP rtsp://mediamtx:8554/mystream/streamid=0 RTSP/1.0
18	1.169021895	172.18.0.2	172.18.0.3	RTSP	241	Reply: RTSP/1.0 200 OK
19	1.169904781	172.18.0.3	172.18.0.2	RTSP	271	SETUP rtsp://mediamtx:8554/mystream/streamid=1 RTSP/1.0
20	1.170150222	172.18.0.2	172.18.0.3	RTSP	241	Reply: RTSP/1.0 200 OK
21	1.170529123	172.18.0.3	172.18.0.2	RTSP	212	RECORD rtsp://mediamtx:8554/mystream RTSP/1.0
26	1.170943194	172.18.0.2	172.18.0.3	RTSP	167	Reply: RTSP/1.0 200 OK
7316	21.302454298	172.18.0.3	172.18.0.2	RTSP	195	TEARDOWN rtsp://mediamtx:8554/mystream RTSP/1.0
7317	21.303089237	172.18.0.2	172.18.0.3	RTSP	113	Reply: RTSP/1.0 200 OK

Figura 1: Flujo de mensajes RTSP

1.4. Software servidor: MediaMTX

MediaMTX es un servidor multimedia que soporta el protocolo RTSP y otros protocolos de streaming. Su función es recibir, administrar y distribuir flujos multimedia en tiempo real.

Para este laboratorio, el servicio fue desplegado en un contenedor Docker construido a partir de un Dockerfile basado en el repositorio oficial del proyecto.

```
carlos@carlos-VMware-Virtual-Platform:~/Lab_Taller_Nedes/servidor$ cat Dockerfile
FROM bluenviron/mediamtx:latest

EXPOSE 8554
```

Figura 2: Dockerfile utilizado para la construcción de la imagen del servidor MediaMTX

Posteriormente, la imagen construida fue utilizada para levantar el servidor RTSP.

```
carlos@carlos-VMware-Virtual-Platform:~/Lab_Taller_Nedes/servidor$ docker run --rm -it --network rtsp-net --name mediamtx -p 8554:8554 mediamtx-server
2026/06/14 08:13:18 INF MediaMTX v1.19.1, linux, amd64
2026/06/14 08:13:18 INF configuration loaded from /mediamtx.yml
2026/06/14 08:13:18 INF [RTSP] started with listeners on :8554 (TCP/RTSP), :8000 (UDP/RTP), :8001 (UDP/RTCP)
2026/06/14 08:13:18 INF [RTMP] started with listener on :1935 (TCP/RTMP)
2026/06/14 08:13:18 INF [HLS] started with listener on :8888 (TCP/HTTP)
2026/06/14 08:13:18 INF [WebRTC] started with listeners on :8889 (TCP/HTTP), :8189 (UDP/ICE)
2026/06/14 08:13:18 INF [SRT] started with listener on :8890 (UDP/SRT)
2026/06/14 08:13:18 WAR [MoQ] certificate auto.key not found, generating it from scratch
2026/06/14 08:13:18 INF [MoQ] started with listeners on :8892 (TCP/HTTP2), :8892 (UDP/HTTP3)
2026/06/14 08:13:29 INF [RTSP] [conn 172.18.0.3:56332] opened
2026/06/14 08:13:29 INF [RTSP] [session 784c5838] created by 172.18.0.3:56332
2026/06/14 08:13:29 INF [path mystream] stream is available and online, 2 tracks (H264, MPEG-4 Audio)
2026/06/14 08:13:29 INF [RTSP] [session 784c5838] is publishing to path 'mystream'
2026/06/14 08:13:29 INF [path mystream] RTP packets are too big (1460 > 1440), remuxing them into smaller ones
2026/06/14 08:14:10 INF [RTSP] [session 784c5838] destroyed: torn down by 172.18.0.3:56332
2026/06/14 08:14:10 INF [RTSP] [conn 172.18.0.3:56332] closed: EOF
```

Figura 3: Servidor MediaMTX ejecutándose en contenedor Docker

1.5. Software cliente: FFmpeg

FFmpeg es una herramienta utilizada para el procesamiento y transmisión de audio y video. En este laboratorio se utilizó como cliente RTSP encargado de enviar un archivo multimedia hacia el servidor.

Al igual que el servidor, se construyó una imagen Docker propia utilizando un Dockerfile basado en el repositorio oficial de FFmpeg.

```
carlos@carlos-VMware-Virtual-Platform:~/Lab_Taller_Nedes/cliente$ cat Dockerfile
FROM jrottenberg/ffmpeg:latest
```

Figura 4: Dockerfile utilizado para la construcción de la imagen del cliente FFmpeg

Una vez construida la imagen, esta fue utilizada para transmitir el video hacia el servidor RTSP.

```
frame= 13 fps=5.2 q=29.0 size=N/A time=00:00:00.36 bitrate[vist#0:0/h264 @ 0x6103fe201c80] Resumed reading at pts 1.066
with rate 1.050 after a lag of 2.035s
[aist#0:1/aac @ 0x6103fe201ac0] Resumed reading at pts 2.041 with rate 1.050 after a lag of 1.442s
frame= 19 fps=6.3 q=29.0 size=N/A time=00:00:00.56 bitrate[aist#0:1/aac @ 0x6103fe201ac0] Resumed reading at pts 2.041
with rate 1.050 after a lag of 1.760s
frame= 29 fps=8.3 q=29.0 size=N/A time=00:00:00.90 bitrate[vist#0:0/h264 @ 0x6103fe201c80] Resumed reading at pts 2.099
with rate 1.050 after a lag of 2.353s
frame= 44 fps= 11 q=29.0 size=N/A time=00:00:01.40 bitrateframe= 50 fps= 11 q=29.0 size=N/A time=00:00:01.60 bitrate[a
vist#0:1/aac @ 0x6103fe201ac0] Resumed reading at pts 3.044 with rate 1.050 after a lag of 2.076s
[aist#0:1/aac @ 0x6103fe201ac0] Resumed reading at pts 3.044 with rate 1.050 after a lag of 2.394s
frame= 62 fps= 12 q=29.0 size=N/A time=00:00:02.00 bitrate[vist#0:0/h264 @ 0x6103fe201c80] Resumed reading at pts 3.132
with rate 1.050 after a lag of 2.673s
frame= 76 fps= 14 q=29.0 size=N/A time=00:00:02.46 bitrate[vist#0:0/h264 @ 0x6103fe201c80] Resumed reading at pts 3.132
with rate 1.050 after a lag of 2.977s
frame= 79 fps= 13 q=29.0 size=N/A time=00:00:02.56 bitrate[aist#0:1/aac @ 0x6103fe201ac0] Resumed reading at pts 4.046
with rate 1.050 after a lag of 2.699s
frame= 89 fps= 14 q=29.0 size=N/A time=00:00:02.90 bitrate[aist#0:1/aac @ 0x6103fe201ac0] Resumed reading at pts 4.046
with rate 1.050 after a lag of 3.016s
[vist#0:0/h264 @ 0x6103fe201c80] Resumed reading at pts 4.165 with rate 1.050 after a lag of 3.285s
```

Figura 5: Cliente FFmpeg transmitiendo video hacia servidor RTSP

1.6. Levantamiento del entorno

Para la implementación del entorno cliente-servidor se construyeron imágenes Docker propias a partir de los Dockerfiles correspondientes a MediaMTX y FFmpeg.

Luego se creó una red Docker para permitir la comunicación entre ambos contenedores:

```
docker network create rtsp-net
```

El servidor MediaMTX fue ejecutado mediante:

```
docker run --rm -it \
--network rtsp-net \
--name mediamtx \
-p 8554:8554 \
mediamtx-server
```

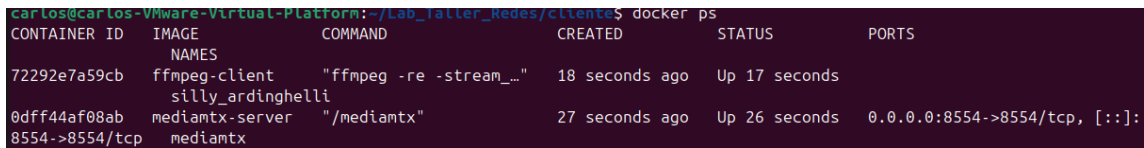
El cliente FFmpeg fue ejecutado utilizando un archivo de video almacenado en el sistema anfitrión:

```
docker run --rm -it \
--network rtsp-net \
--name ffmpeg-client \
-v ~/Escritorio/Trabajotallerderedes:/videos \
ffmpeg-client \
-re -stream_loop -1 \
-i /videos/video.mp4 \
-c:v libx264 -preset veryfast \
-c:a aac \
```

```
-f rtsp \
rtsp://172.17.0.1:8554/mystream
```

La correcta ejecución de ambos contenedores fue verificada mediante el comando:

```
docker ps
```



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
72292e7a59cb	ffmpeg-client	"ffmpeg -re -stream_..."	18 seconds ago	Up 17 seconds	
0dff44af08ab	mediamtx-server	"/mediamtx"	27 seconds ago	Up 26 seconds	0.0.0.0:8554->8554/tcp, [::]:8554->8554/tcp

Figura 6: Verificación de contenedores activos mediante docker ps

1.7. Descripción de los parámetros utilizados

Para la transmisión del video mediante FFmpeg se emplearon los siguientes parámetros:

- **-v /Escritorio/Trabajotallerderedes:/videos:** monta una carpeta local dentro del contenedor para acceder al archivo de video.
- **-re:** lee el video en tiempo real simulando una transmisión en vivo.
- **-stream_loop -1:** repite el video indefinidamente.
- **-i /videos/video.mp4:** especifica el archivo de entrada.
- **-c:v libx264:** codifica el video en H.264.
- **-preset veryfast:** reduce el uso de recursos en la codificación.
- **-c:a aac:** codifica el audio en AAC.
- **-f rtsp:** define el protocolo de salida como RTSP.
- **rtsp://172.17.0.1:8554/mystream:** dirección del servidor RTSP.

1.8. Suposición de fallo

Una posible causa de fallo corresponde a la pérdida de comunicación entre cliente y servidor. Si el cliente FFmpeg deja de transmitir datos, el servidor MediaMTX detecta la ausencia de flujo y finaliza la sesión mediante timeout o EOF.

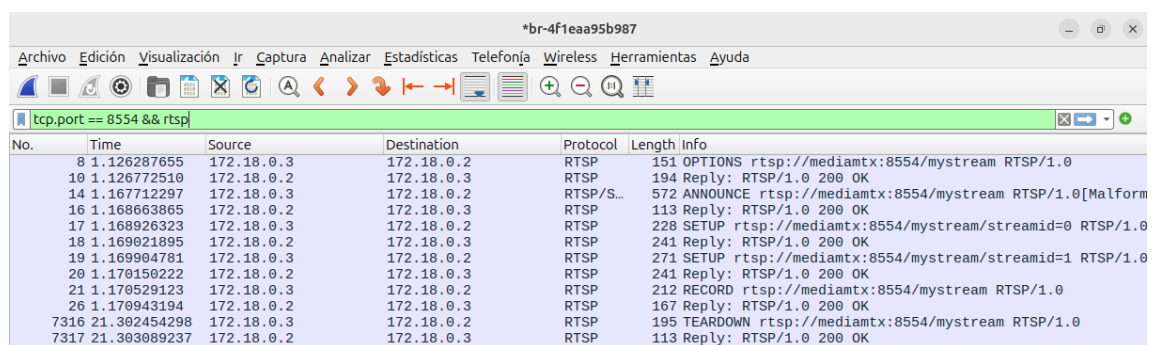
Otra causa de error puede ser una configuración incorrecta de la interfaz de red en Docker. Durante el desarrollo del laboratorio inicialmente se intentó capturar tráfico en la interfaz `docker0`, sin embargo no se observaron paquetes RTSP. Posteriormente se utilizó la interfaz bridge de Docker, donde fue posible visualizar correctamente la comunicación entre cliente y servidor.

2. Capítulo II: Análisis de tráfico

2.1. Herramienta utilizada

Para realizar el análisis del tráfico generado por el servicio RTSP se utilizó Wireshark, herramienta que permite capturar y examinar los paquetes intercambiados entre cliente y servidor.

La captura fue realizada sobre la interfaz bridge correspondiente a la red utilizada por Docker, permitiendo observar correctamente la comunicación entre ambos contenedores.



No.	Time	Source	Destination	Protocol	Length	Info
8	1.126287655	172.18.0.3	172.18.0.2	RTSP	151	OPTIONS rtsp://mediamtx:8554/mystream RTSP/1.0
10	1.126772510	172.18.0.2	172.18.0.3	RTSP	194	Reply: RTSP/1.0 200 OK
14	1.167712297	172.18.0.3	172.18.0.2	RTSP/S...	572	ANNOUNCE rtsp://mediamtx:8554/mystream RTSP/1.0[Malform
16	1.168663865	172.18.0.2	172.18.0.3	RTSP	113	Reply: RTSP/1.0 200 OK
17	1.168926323	172.18.0.3	172.18.0.2	RTSP	228	SETUP rtsp://mediamtx:8554/mystream/streamid=0 RTSP/1.0
18	1.169021895	172.18.0.2	172.18.0.3	RTSP	241	Reply: RTSP/1.0 200 OK
19	1.169904781	172.18.0.3	172.18.0.2	RTSP	271	SETUP rtsp://mediamtx:8554/mystream/streamid=1 RTSP/1.0
20	1.170150222	172.18.0.2	172.18.0.3	RTSP	241	Reply: RTSP/1.0 200 OK
21	1.170529123	172.18.0.3	172.18.0.2	RTSP	212	RECORD rtsp://mediamtx:8554/mystream RTSP/1.0
26	1.170943194	172.18.0.2	172.18.0.3	RTSP	167	Reply: RTSP/1.0 200 OK
7316	21.302454298	172.18.0.3	172.18.0.2	RTSP	195	TEARDOWN rtsp://mediamtx:8554/mystream RTSP/1.0
7317	21.303089237	172.18.0.2	172.18.0.3	RTSP	113	Reply: RTSP/1.0 200 OK

Figura 7: Captura realizada con Wireshark sobre la interfaz de red utilizada por Docker

2.2. Filtrado de tráfico

Para aislar únicamente los paquetes correspondientes al protocolo RTSP se empleó el siguiente filtro:

```
tcp.port == 8554 && rtsp
```

Este filtro permitió visualizar exclusivamente las solicitudes y respuestas asociadas al servicio implementado.

2.3. Análisis del tráfico RTSP

Durante la captura realizada con Wireshark fue posible observar el intercambio de distintos mensajes RTSP entre el cliente FFmpeg y el servidor MediaMTX.

- **OPTIONS**: permite consultar las capacidades soportadas por el servidor.
- **ANNOUNCE**: utilizado por el cliente para anunciar la publicación del flujo multimedia.
- **SETUP**: establece los parámetros de la sesión y configura los recursos necesarios para la transmisión.

- **RECORD**: indica el comienzo del envío del flujo multimedia hacia el servidor.
- **200 OK**: respuesta enviada por el servidor para confirmar que las solicitudes fueron procesadas correctamente.
- **TEARDOWN**: finaliza la sesión RTSP y libera los recursos asociados.

La captura permitió comprobar el comportamiento esperado del protocolo RTSP y verificar la correcta comunicación entre el cliente FFmpeg y el servidor MediaMTX.

No.	Time	Source	Destination	Protocol	Length	Info
17	1.168926323	172.18.0.3	172.18.0.2	RTSP	228	SETUP rtsp://mediamtx:8554/mystream/streamid=0 RTSP/1.0
18	1.169021895	172.18.0.2	172.18.0.3	RTSP	241	Reply: RTSP/1.0 200 OK
19	1.169904781	172.18.0.3	172.18.0.2	RTSP	271	SETUP rtsp://mediamtx:8554/mystream/streamid=1 RTSP/1.0
20	1.170150222	172.18.0.2	172.18.0.3	RTSP	241	Reply: RTSP/1.0 200 OK
21	1.170529123	172.18.0.3	172.18.0.2	RTSP	212	RECORD rtsp://mediamtx:8554/mystream RTSP/1.0
26	1.170943194	172.18.0.2	172.18.0.3	RTSP	167	Reply: RTSP/1.0 200 OK
7316	21.302454298	172.18.0.3	172.18.0.2	RTSP	195	TEARDOWN rtsp://mediamtx:8554/mystream RTSP/1.0
7317	21.303089237	172.18.0.2	172.18.0.3	RTSP	113	Reply: RTSP/1.0 200 OK

Frame 17: 228 bytes on wire (1824 bits), 228 bytes captured (1824 bits) on interface br-4f1eaa95b987, id 0
Ethernet II, Src: 32:1e:8f:1b:b1:53 (32:1e:8f:1b:b1:53), Dst: 9a:5c:ae:d9:0c:32 (9a:5c:ae:d9:0c:32)
Internet Protocol Version 4, Src: 172.18.0.3, Dst: 172.18.0.2
Transmission Control Protocol, Src Port: 32814, Dst Port: 8554, Seq: 730, Ack: 176, Len: 162
Real Time Streaming Protocol
Request: SETUP rtsp://mediamtx:8554/mystream/streamid=0 RTSP/1.0\r\n
Method: SETUP
URL: rtsp://mediamtx:8554/mystream/streamid=0
Transport: RTP/AVP/UDP;unicast;client_port=20922-20923;mode=record
CSeq: 3\r\n
User-Agent: Lavf62.3.100\r\n
\r\n

Figura 8: Detalle de un paquete RTSP capturado mediante Wireshark

2.4. Comportamiento de la sesión

La sesión RTSP se mantiene activa mientras el cliente FFmpeg continúa transmitiendo el archivo multimedia. Durante este período el servidor MediaMTX recibe el flujo y mantiene abierta la sesión establecida.

Cuando el cliente deja de enviar información, ya sea por la finalización del proceso o por una interrupción de la transmisión, MediaMTX detecta la ausencia de datos y procede a cerrar la sesión mediante EOF o timeout, liberando los recursos asociados.

```
2026/06/12 07:36:47 INF [RTSP] [session 7ee70bc7] created by 172.18.0.3:40652
2026/06/12 07:36:47 INF [path mystream] stream is available and online, 2 tracks (H264, MPEG-4 Audio)
2026/06/12 07:36:47 INF [RTSP] [session 7ee70bc7] is publishing to path 'mystream'
```

Figura 9: Sesión RTSP activa observada desde MediaMTX

2.5. Posibles fallos en la comunicación

Si el cliente FFmpeg deja de transmitir el flujo multimedia o se produce una interrupción en la red entre ambos contenedores, la sesión RTSP dejará de recibir datos y el servidor MediaMTX terminará cerrando la conexión mediante timeout.

De igual forma, una configuración incorrecta de la interfaz utilizada para la captura puede impedir la visualización del tráfico RTSP. Durante el desarrollo del laboratorio fue necesario identificar la interfaz bridge utilizada por Docker, ya que inicialmente se intentó capturar tráfico sobre la interfaz `docker0`, donde no fue posible observar los paquetes intercambiados entre cliente y servidor.

3. Conclusiones

Durante el desarrollo de este laboratorio se implementó una arquitectura cliente-servidor basada en el protocolo RTSP utilizando contenedores Docker. Para ello se empleó MediaMTX como servidor y FFmpeg como cliente encargado de transmitir un archivo multimedia.

A partir de los repositorios oficiales de ambos proyectos se construyeron imágenes Docker propias mediante Dockerfiles, permitiendo implementar el entorno de pruebas requerido y establecer correctamente la comunicación entre cliente y servidor.

Mediante el uso de Wireshark fue posible capturar y analizar el tráfico generado por la transmisión, observando mensajes característicos del protocolo RTSP como OPTIONS, ANNOUNCE, SETUP, RECORD, TEARDOWN y las respuestas 200 OK, comprobando así el funcionamiento normal de una sesión RTSP.

Además, durante el desarrollo se identificaron distintos problemas asociados a la selección de las interfaces de red utilizadas por Docker y al establecimiento de la comunicación entre los contenedores, evidenciando la importancia de una correcta configuración del entorno de pruebas.

Finalmente, el laboratorio permitió adquirir experiencia práctica en la implementación de servicios de streaming, en el análisis de protocolos de red y en la utilización de herramientas como Docker y Wireshark para estudiar el comportamiento del tráfico en una arquitectura cliente-servidor.

4. Bibliografía

- IETF. *RFC 2326: Real Time Streaming Protocol (RTSP)*. 1998.
- FFmpeg Project. <https://github.com/FFmpeg/FFmpeg>
- MediaMTX Project. <https://github.com/bluenviron/mediamtx>
- Kurose, J. F., & Ross, K. W. *Computer Networking: A Top-Down Approach*. Pearson.
- Wireshark Foundation. <https://www.wireshark.org>
- Docker Inc. <https://www.docker.com>

-
- Video de la tarea: <https://youtu.be/UxPs5rSPfo8>